Use Cases:

UC-01 Start or reset the planning workspace

Item	Details
Requirement	FR-03
Goal	Pick up from saved data, or start from a documented sample dataset, without ever losing data silently.
Trigger	The organizer starts CampusPlan.
Success end condition	The room catalogue and all saved records are loaded with their original IDs.
Failure end condition	CampusPlan does not run with bad data, and saved data is not changed during startup unless the organizer confirms it.

Main flow:

1. The organizer starts CampusPlan.
2. The system loads the room catalogue from seed/rooms.json. It checks every room and makes sure no two rooms share an ID or a name (ignoring case).
3. The system loads the saved students, availability and draft events. They are checked against the same rules as typed input, and every reference (for example an event's invitees) has to point to a record that exists.
4. The system shows the main menu.

Alternate flows:

- 3a. There is no saved data yet (first use). The system says no saved data was found and offers to either load a bundled sample dataset or start with no students. There is one sample where a meeting can be scheduled and one where it cannot. Nothing is written to disk until the first confirmed change.
- 3b. Some saved intervals overlap or touch. They are merged while loading.

- R1. The organizer resets during a session by choosing to load sample data. The system lists what will be replaced and asks the organizer to type YES. Any other answer leaves everything as it was.

Exceptions:

- 2a. The catalogue file is missing. The system prints "Error: Room catalogue not found: /seed/rooms.json" and exits with status 1.
- 2b. The catalogue is invalid (malformed or empty JSON, a capacity outside 1-100, an unknown feature, or a duplicate ID or name). The system prints "Error: Room catalogue is invalid: " followed by the problem, for example "duplicate room ID R01.", and exits with status 1.
- 3c. The saved data can't be read or isn't valid JSON. The system prints an `Error:` line naming the file and where it is broken, then gives two options: quit (the default), or set the file aside and start empty. Setting it aside needs explicit confirmation, and the old file is kept under a new name.
- 3d. The saved data is valid JSON but breaks a rule, such as a duplicate email or an invitee who doesn't exist. This is handled the same way as 3c, and each problem is listed with its location in the file.
- 3e. The saved data was written by a newer version of CampusPlan. The system refuses to open it and exits without changing anything.

UC-02 Register and maintain students

Item	Details
Requirement	FR-01
Goal	Keep a single, clean list of students who can be invited, with duplicates caught by email.
Trigger	The organizer chooses 1 (Add), 2 (Edit), 3 (View) or 4 (Search).
Success end condition	The student is created or updated. Their ID does not change.
Failure end condition	No student records change and no ID is used up.

Main flow (adding a student):

1. The organizer chooses 1 (Add student).
2. At the `Name:` prompt the organizer enters "Farah Haddad", and at `Email:` they enter "Farah.Haddad@campus.example".
3. The system trims the name, trims and lowercases the email, and validates both.
4. The system checks that no other student already uses that email.
5. The system assigns the next ID, stores the student and prints "Created S013 Farah Haddad farah.haddad@campus.example".

Alternate flows:

- A1. Search (option 4). At `Name or email contains:` the organizer enters "had". The system lists every student whose name or email contains that text, ignoring case, in the order they were added. Leaving it blank lists everyone.
- A2. View (option 3). The organizer enters an ID at `Student ID:` (extra spaces are ignored), and the system prints the student's details, starting with their ID, name and email.
- A3. Edit (option 2). The organizer enters the student's ID and the system prints "Editing S013 Farah Haddad <...> (leave blank to keep current value)". It then asks `Name [Farah Haddad]:` and `Email [farah.haddad@campus.example]:`, where a blank answer keeps the current value. The new values are checked the same way as in steps 3 and 4, except that the student's own email doesn't count as a duplicate. The system saves the change and prints "Updated ...". The ID stays the same.
- A3a. If only the capitalization of the email changes, the edit is accepted, since emails are stored in lowercase anyway.

Exceptions:

- 3a. The name is empty after trimming or longer than 100 characters. The system prints "Error: Student name must be 1-100 characters after trimming."
- 3b. The email is invalid. The system prints "Error: Email must contain exactly one @, text on both sides, and no whitespace."
- 4a. The email is already taken. The system prints "Error: Email farah.haddad@campus.example is already used by another student."
- A1a. The search has no matches. The system prints "No students found."
- A2a / A3b. The ID doesn't exist. The system prints "Error: No student with ID S099." When editing, this happens before any fields are asked for.
- 5a. The save fails. The change isn't applied and the system prints an `Error:` line with the reason (see UC-07).

UC-03 Find rooms that meet a group's requirements

Item	Details
Requirement	FR-01
Goal	See which rooms work for a group without having to check a separate room list.
Trigger	The organizer chooses 5 (List rooms) or 6 (Filter rooms).
Success end condition	The matching rooms are listed by ID, along with the capacity and features that make each one qualify.
Failure end condition	Nothing changes, since this use case only reads data.

Main flow:

1. The organizer chooses 6 (Filter rooms).
2. At `Minimum capacity (blank for any):` the organizer enters 6.
3. At `Required features, comma-separated [WHITEBOARD, PROJECTOR, ACCESSIBLE] (blank for none):` the organizer enters whiteboard.
4. The system prints a table with the ID, name, capacity and features of every room that seats at least 6 people and has a whiteboard, sorted by ID. With the seed catalogue this is R01, R02, R04 and R06.

Alternate flows:

- 1a. The organizer chooses 5 (List rooms) instead. The system prints the same table for every room. Rooms with no features show a dash.
- 2a. The capacity is left blank, so any capacity matches.
- 3a. The features are left blank, so no features are required.
- 3b. Features are typed in a different case, with extra spaces, or more than once (for example "Whiteboard, whiteboard"). They are treated as a single requirement.
- 4a. No rooms match. The system prints "No rooms match."

Exceptions:

- 2b. The capacity isn't a whole number (for example "lots"). The system prints "Error: Minimum capacity must be a whole number."

- 2c. The capacity is less than 1 (for example 0 or -3). The system prints "Error: Minimum capacity must be at least 1."
- 3c. A feature isn't recognized (for example "sauna"). The system prints "Error: Unknown room feature: sauna."

UC-04 Record a student's availability

Item	Details
Requirement	FR-02
Goal	Turn what students say about their free time into exact intervals, and update them when plans change.
Trigger	The organizer chooses to add or remove availability.
Success end condition	The student's intervals are stored merged, so none overlap or touch on the same date.
Failure end condition	The student's availability does not change.

Main flow (adding free time):

1. The organizer chooses to add availability.
2. The organizer enters the student ID (S001), the date (2026-11-03), the start time (13:00) and the end time (15:30). Dates use YYYY-MM-DD and times use HH:MM.
3. The system validates the interval.
4. The system merges it with any interval on the same date that it overlaps or touches. For example, if S001 already had 15:30-17:00, the result is 13:00-17:00.
5. The system confirms the change and shows the student's availability for that date.

Alternate flows:

- 1a. The organizer removes free time instead, entering the same four values (for example 14:00-15:00). The system removes that range and splits an interval if it has to, so 13:00-17:00 becomes 13:00-14:00 and 15:00-17:00.
- 4a. The new interval is already covered, for example adding 13:30-14:00 when the student has 13:00-17:00. Nothing changes and the system tells the organizer.
- 4b. Intervals on different dates are never merged.

- A1. Viewing a student (UC-02, A2) also shows their availability grouped by date.

Exceptions (each one prints an `Error:` line explaining the rule, and nothing changes):

- 2a. The student doesn't exist. The message is the same one used in UC-02: "Error: No student with ID S099."
- 3a. The date or time is in the wrong format, for example "3/11" or "1pm".
- 3b. A time isn't on the hour or half hour, for example 13:15.
- 3c. The start isn't before the end, for example 15:00-15:00.
- 3d. The interval is outside opening hours, for example 07:30-09:00.
- 1a-1. There is nothing to remove because the student has no availability in that range.

UC-05 Set up a draft event

Item	Details
Requirement	FR-02
Goal	Capture a planning request as a single record that P2 can schedule from.
Trigger	The organizer chooses to create a draft event.
Success end condition	The event is stored with a new ID and DRAFT status. Every invitee is PENDING, and no room or time has been assigned.
Failure end condition	No event is created and no ID is used up.

Main flow:

1. The organizer chooses to create a draft event.
2. The organizer enters:
    - the title (Project meeting)
    - the invitee IDs, separated by commas (S001, S002, S003, S004, S005, S006)
    - the first and last date (2026-11-02 and 2026-11-06)
    - the duration in minutes (120)
    - the required features, in the same format as the room filter (whiteboard)
    - the minimum attendance (4)
3. The system validates each value and checks that the values make sense together.

4. The system creates the draft and shows its ID and a summary. It also reports how many rooms in the catalogue can seat all the invitees and have the required features, which is 4 in this example.

Alternate flows:

- 2a. The features are left blank, or the window is a single day (same first and last date). Both are allowed.
- 4a. No room fits the event. The draft is still created, but the system warns that scheduling won't be able to find any suggestions.

Exceptions (each one prints an `Error:` line explaining the rule, nothing is created, and the menu is shown again):

- The title is empty or longer than 100 characters.
- An invitee doesn't exist: "Error: No student with ID S099."
- The same invitee is listed twice. The message names the repeated ID.
- There are fewer than 2 or more than 20 invitees.
- The last date is before the first date, or the window is longer than 7 days.
- The duration is outside 30-180 minutes or isn't a multiple of 30.
- A feature isn't recognized. This uses the same message as the room filter: "Error: Unknown room feature: sauna."
- The minimum attendance is less than 2 or more than the number of invitees. The message gives the allowed range.

UC-06 Revise a draft event when plans change

Item	Details
Requirement	FR-02
Goal	Update a draft in one edit instead of starting over, for example when someone joins, the week changes or a projector is needed.
Trigger	The organizer chooses to edit, view or list events.
Success end condition	The change is saved. The ID and status stay the same, and existing invitees keep their responses.
Failure end condition	The event is left exactly as it was.

Main flow:

1. The organizer chooses to edit an event and enters its ID.
2. The system asks for each of the UC-05 values in turn, the same way it does when editing a student: the current value is shown in brackets and a blank answer keeps it.
3. The organizer adds S007 to the invitee list and leaves everything else blank.
4. The system validates the whole event with the changes applied.
5. The system saves the change and shows the updated event, with S007 as PENDING.

Alternate flows:

- 3a. An invitee is removed from the list. Their response is removed as well.
- 3b. Other fields are changed (window, duration, features, minimum or title). Each one is checked using the UC-05 rules.
- A1. The organizer views an event. The system shows all of its fields, each invitee's name and response, and its status.
- A2. The organizer lists events. The system shows each event's ID, title, status, window and number of invitees, sorted by first date and then by ID.

Exceptions (each one prints an `Error:` line explaining the rule, and nothing changes):

- 1a. The event ID doesn't exist.
- 1b. The event isn't a draft. Only DRAFT events can be edited, and the sample data may include events in other states.

- 4a. The values contradict each other, for example removing 2 of 6 invitees when the minimum is 5.
- 4b. One of the UC-05 field rules is broken.

UC-07 Keep work safe between sessions

Item	Details
Requirement	FR-03
Goal	Save every confirmed change straight away, so the next session shows the same records with the same IDs.
Trigger	The organizer confirms a change in UC-02, UC-04, UC-05 or UC-06, or does a reset in UC-01.
Success end condition	Students, availability and events are stored as UTF-8 JSON, and reloading gives back identical records and IDs.
Failure end condition	The saved data always holds the last complete save, never a half-written one.

Main flow:

1. The organizer confirms a change, such as adding Farah Haddad.
2. The system saves all of the data in a way that either fully succeeds or leaves the previous save untouched.
3. The system prints the success message ("Created S013 ...").
4. The organizer quits. In the next session Farah is still S013, and the next student added gets S014.

Alternate flows:

- 1a. Nothing actually changed, so nothing is written.
- 2a. The data folder doesn't exist yet, so it is created.
- 4a. The window is closed or input ends without choosing Quit. Nothing is lost, because every change was already saved.

Exceptions:

- 2b. The save fails (for example the disk is full or there's no permission). The system prints an `Error:` line with the reason and the change is not applied. The session goes back to how it was before the change, and the previous save is untouched.
- 2c. CampusPlan crashes in the middle of a save. The previous complete save is loaded the next time it starts.
- 4b. The saved data was damaged between sessions. This is handled by UC-01 (exceptions 3c and 3d) at the next start.

Acceptance Criteria:

AC-01 Start or reset the workspace (UC-01)

User story: As the organizer, I want CampusPlan to open with my saved work, or with sample data the first time I use it, so that I can pick up where I left off without losing anything.

- AC-01.1 On startup, all 7 rooms from the seed catalogue are available and listed in ID order.
- AC-01.2 If the room catalogue file is missing, CampusPlan prints "Error: Room catalogue not found: /seed/rooms.json" and closes with exit status 1 without showing the menu.
- AC-01.3 If two rooms in the catalogue share an ID, CampusPlan prints an error starting with "Error: Room catalogue is invalid:" that names the duplicate ID, and closes.
- AC-01.4 If two room names in the catalogue differ only in capitalization, CampusPlan reports the catalogue as invalid and closes.
- AC-01.5 If a room's capacity is below 1 or above 100, CampusPlan reports the catalogue as invalid and closes.
- AC-01.6 If a room's name is blank or longer than 60 characters, CampusPlan reports the catalogue as invalid and closes.
- AC-01.7 If a room lists a feature other than WHITEBOARD, PROJECTOR or ACCESSIBLE, CampusPlan reports the catalogue as invalid and closes.
- AC-01.8 If the catalogue file is empty or is not valid JSON, CampusPlan reports the catalogue as invalid and closes.
- AC-01.9 When there is no saved data, the organizer is told so and can choose a sample dataset or an empty start.
- AC-01.10 When there is no saved data, no data file is created until the organizer confirms their first change.

- AC-01.11 Loading the sample where a meeting can be scheduled gives at least 12 students, 7 days of availability and 3 events.
- AC-01.12 Each bundled sample dataset loads without any errors.
- AC-01.13 If the saved data file is cut off or is not valid JSON, the organizer sees an error naming the file and the line and column where it breaks, and the file is not modified.
- AC-01.14 After a corrupt-file error, choosing to quit closes CampusPlan and leaves the file exactly as it was.
- AC-01.15 After a corrupt-file error, confirming that the file should be set aside keeps it under a new name with identical contents and opens an empty session.
- AC-01.16 If saved data refers to a student that does not exist (for example an event invitee), the organizer sees an error naming the problem record and the data is not loaded.
- AC-01.17 If the saved data was written by a newer version of CampusPlan, it refuses to open it, explains why, and leaves the file exactly as it was.
- AC-01.18 Overlapping or touching availability in saved data is shown as a single merged interval after loading.
- AC-01.19 Starting a reset to sample data and answering anything other than YES leaves all current data unchanged.
- AC-01.20 Confirming a reset with YES replaces the current data with the chosen sample dataset.

AC-02 Register and maintain students (UC-02)

User story: As the organizer, I want to add, edit, view and search students so that I have one accurate list of the people I can invite.

- AC-02.1 Adding a student with a valid name and email prints "Created" followed by the new ID, name and email, for example "Created S001 Ada Lovelace ada@example.com".
- AC-02.2 The first student gets ID S001, and each new student gets the next number (S002, S003 and so on).
- AC-02.3 Spaces before and after the name and email are removed before the student is saved.
- AC-02.4 Emails are saved in lowercase, so "Ada@Example.COM" is stored as "ada@example.com".
- AC-02.5 Names that are 1 or 100 characters long are accepted.

- AC-02.6 A name that is empty, only spaces, or longer than 100 characters is rejected with "Error: Student name must be 1-100 characters after trimming."
- AC-02.7 An email that does not have exactly one @, has nothing before or after the @, or contains a space or tab is rejected with "Error: Email must contain exactly one @, text on both sides, and no whitespace."
- AC-02.8 Emails such as a@b and first.last+tag@sub.example.com are accepted.
- AC-02.9 Adding a student whose email matches an existing student's, ignoring case and surrounding spaces, is rejected with "Error: Email ada@example.com is already used by another student."
- AC-02.10 When adding a student is rejected, no student is created and no ID is used up.
- AC-02.11 Viewing a student by ID shows their ID, name and email, and any spaces around the ID are ignored.
- AC-02.12 Viewing or editing an ID that does not exist prints "Error: No student with ID S404." (with the ID entered) and shows no edit prompts.
- AC-02.13 When editing, each prompt shows the current value in brackets, and leaving a prompt blank keeps that value.
- AC-02.14 A successful edit prints "Updated" followed by the student's ID, name and email, and the ID is unchanged.
- AC-02.15 Changing a student's email to one used by another student is rejected, and neither student changes.
- AC-02.16 Changing only the capitalization of a student's own email is accepted.
- AC-02.17 An edit with an invalid name or email is rejected, and the student is unchanged.
- AC-02.18 Searching lists every student whose name or email contains the search text, ignoring case.
- AC-02.19 Search results are listed in the order the students were added.
- AC-02.20 A blank search lists all students.
- AC-02.21 A search with no matches prints "No students found."
- AC-02.22 Entering a menu option that does not exist prints "Unknown option:" followed by what was entered, and the menu is shown again.
- AC-02.23 Choosing 0, or reaching the end of input, prints "Goodbye." and closes CampusPlan.

AC-03 Find rooms (UC-03)

User story: As the organizer, I want to filter the room catalogue by group size and features so that I can see which rooms fit a group without checking a separate list.

- AC-03.1 Listing rooms shows every room in the catalogue in ID order, with its name, capacity and features.
- AC-03.2 A room with no features shows "-" in the features column.
- AC-03.3 Filtering by a minimum capacity shows only rooms that seat at least that many people. A minimum of 40 shows only R05.
- AC-03.4 Filtering by features shows only rooms that have every feature listed. Requiring all three features shows only R02.
- AC-03.5 Filtering for a capacity of 6 and a whiteboard shows R01, R02, R04 and R06, in that order.
- AC-03.6 Leaving the capacity blank applies no capacity limit.
- AC-03.7 Leaving the features blank applies no feature requirement.
- AC-03.8 Feature names match regardless of capitalization or repetition, so "Projector, projector" shows R02, R05 and R06.
- AC-03.9 When no rooms match, CampusPlan prints "No rooms match." A minimum capacity of 41 gives this result.
- AC-03.10 A capacity that is not a whole number is rejected with "Error: Minimum capacity must be a whole number."
- AC-03.11 A capacity below 1 is rejected with "Error: Minimum capacity must be at least 1."
- AC-03.12 An unrecognized feature is rejected with "Error: Unknown room feature:" followed by the name entered, for example "Error: Unknown room feature: sauna."

AC-04 Record availability (UC-04)

User story: As the organizer, I want to record when each student is free so that I can later see who is able to attend a proposed meeting.

- AC-04.1 Adding a valid interval for a student saves it and shows that student's availability for the date.
- AC-04.2 Adding an interval that overlaps an existing one on the same date combines them, so 13:00-15:00 plus 14:00-16:00 becomes 13:00-16:00.
- AC-04.3 Adding an interval that touches an existing one combines them, so 13:00-15:00 plus 15:00-16:00 becomes 13:00-16:00.

- AC-04.4 Intervals with a gap between them stay separate, so 13:00-15:00 and 15:30-16:00 are kept as two intervals.
- AC-04.5 Intervals on different dates are never combined.
- AC-04.6 Adding an interval that is already fully covered tells the organizer that nothing changed, and the availability stays the same.
- AC-04.7 Removing part of an interval keeps the rest, so removing 14:00-15:00 from 13:00-17:00 leaves 13:00-14:00 and 15:00-17:00.
- AC-04.8 Removing a range where the student has no availability is rejected with an error, and nothing changes.
- AC-04.9 A time that is not on the hour or half hour, such as 13:15, is rejected.
- AC-04.10 An interval whose start is not before its end, such as 15:00-15:00 or 16:00-15:00, is rejected.
- AC-04.11 An interval that starts before 08:00 or ends after 22:00 is rejected.
- AC-04.12 An interval of exactly 08:00-22:00 is accepted.
- AC-04.13 A date not in YYYY-MM-DD form, or a time not in HH:MM form, is rejected.
- AC-04.14 Adding availability for a student ID that does not exist prints "Error: No student with ID S099." (with the ID entered).
- AC-04.15 Viewing a student shows their availability grouped by date.

AC-05 Set up a draft event (UC-05)

User story: As the organizer, I want to create a draft event with its invitees, date window, duration, room features and minimum turnout so that the request is captured in one place and ready to be scheduled.

- AC-05.1 Creating a valid draft shows the new event's ID and a summary of the values entered.
- AC-05.2 A new draft has DRAFT status, every invitee is PENDING, and no room or time is assigned.
- AC-05.3 After a draft is created, the organizer is told how many catalogue rooms can seat every invitee and have the required features. For 6 invitees needing a whiteboard, this is 4.
- AC-05.4 If no room fits, the draft is still created and a warning says scheduling will not find any suggestions. This happens for 20 invitees needing WHITEBOARD and ACCESSIBLE.
- AC-05.5 A draft with no required features is accepted.

- AC-05.6 A one-day window, where the first and last dates are the same, is accepted.
- AC-05.7 A title that is empty or longer than 100 characters is rejected.
- AC-05.8 A draft with fewer than 2 or more than 20 invitees is rejected.
- AC-05.9 An invitee listed more than once is rejected with an error naming the repeated ID.
- AC-05.10 An invitee ID that does not exist is rejected with "Error: No student with ID S099." (with the ID entered).
- AC-05.11 A date window of 7 days is accepted, and a window of 8 days or more is rejected.
- AC-05.12 A last date that comes before the first date is rejected.
- AC-05.13 Durations of 30 and 180 minutes are accepted, and durations of 0, 45 and 210 minutes are rejected.
- AC-05.14 With 6 invitees, a minimum attendance of 2 or 6 is accepted, and 1 or 7 is rejected with an error giving the allowed range.
- AC-05.15 When a draft is rejected, no event is created and no event ID is used up.

AC-06 Revise a draft event (UC-06)

User story: As the organizer, I want to edit a draft event when plans change so that I do not have to recreate it from scratch.

- AC-06.1 Listing events shows each event's ID, title, status, date window and number of invitees, sorted by first date and then by ID.
- AC-06.2 Viewing an event shows all of its fields, each invitee's name and response, and its status.
- AC-06.3 When editing, each prompt shows the current value in brackets, and leaving a prompt blank keeps that value.
- AC-06.4 An invitee added to a draft starts as PENDING, and existing invitees keep their responses.
- AC-06.5 Removing an invitee from a draft also removes their response.
- AC-06.6 Removing invitees so that fewer remain than the minimum attendance is rejected, and the event is unchanged.
- AC-06.7 Adding a 21st invitee is rejected, and the event is unchanged.
- AC-06.8 Trying to edit an event that is not a DRAFT is rejected with an error saying only DRAFT events can be edited.

- AC-06.9 Viewing or editing an event ID that does not exist is rejected with an error naming the ID.
- AC-06.10 After an accepted edit, the event keeps the same ID and DRAFT status.
- AC-06.11 An edit that breaks any of the AC-05 rules for titles, invitees, dates, duration, features or minimum attendance is rejected, and the event is unchanged.

AC-07 Keep work safe between sessions (UC-07)

User story: As the organizer, I want every change saved as soon as I confirm it so that my work is still there the next time I open CampusPlan.

- AC-07.1 After closing and reopening CampusPlan, every student, availability interval and draft event is shown with the same details and the same ID.
- AC-07.2 After a restart, the next student added gets the next unused number, so S013 if the highest existing ID is S012.
- AC-07.3 Names with accented or non-Latin characters, such as Zoë Ñúñez 李, appear exactly the same after a restart.
- AC-07.4 A change is saved before its success message appears, so closing CampusPlan immediately afterwards does not lose it.
- AC-07.5 If CampusPlan stops partway through a save, the last complete save is loaded the next time it starts.
- AC-07.6 If a save fails, for example because the disk is full or the folder is read-only, an error is shown, the change is not applied, and the previous save is untouched.
- AC-07.7 Confirming an action that changes nothing does not rewrite the saved data.
- AC-07.8 In a scripted session that adds a student, quits, starts CampusPlan again and searches for that student, the student is found.

Non-Functional Requirements:

Our top priorities are reliability, usability and maintainability, because the biggest risks for the organizer are losing data, getting stuck on unclear errors, and code that is too hard to extend in P2 and P3. Performance and portability still have targets, but they matter less for a single-user program with a small dataset on one computer.

Tests that check an NFR include its ID in their name. If a threshold changes, the change and the reason are recorded in the repository.

ID	Label	Quality	Requirement	Applies to	How we verify it
NFR-01	Rejected input changes nothing	Reliability	When the organizer enters a value that breaks any rule in the use cases, CampusPlan shall reject the whole task, leave every stored record unchanged and not use up an ID. This must hold in 100% of the rejection cases in section 2.	UC-02, UC-04, UC-05, UC-06	Automated tests compare the stored data before and after every rejection case in section 2
NFR-02	Bad data never loads	Reliability	When CampusPlan starts with a room catalogue or saved data file that is missing, malformed or breaks a rule, it shall not show the main menu with that data, and the file shall stay byte-for-byte unchanged unless the organizer types YES.	UC-01	Tests take a checksum of the file before and after startup for each case in AC-01.2 to AC-01.8 and AC-01.13 to AC-01.17
NFR-03	Confirmed work survives a crash	Reliability	When the organizer confirms a change, CampusPlan shall write it to disk before printing the success message. If the process is killed at any point after that message, 100% of confirmed changes shall be there on the next start. If it is killed during a save, the last complete save shall load.	UC-07	AC-07.4, AC-07.5, and a manual test that kills the process 20 times across all options that change data

ID	Label	Quality	Requirement	Applies to	How we verify it
NFR-04	First-time users can work unaided	Usability	When a first-time user has only the menu to go on, at least 4 out of 5 test users (the client and four classmates outside the team) shall complete each of UAT-1 to UAT-3 on their first attempt, without help, in 5 minutes or less per task.	UC-02 to UC-06	A timed usability session at the end of each iteration, with results recorded in the repository
NFR-05	Error messages are short and specific	Usability	When input is rejected, CampusPlan shall print exactly one line that starts with "Error:", names the rule that was broken, and is 120 characters or fewer (not counting any value the organizer typed), followed by the menu.	All use cases	An automated test checks the prefix, line count and length for every rejection case in section 2, plus UAT-5
NFR-06	The CLI never crashes on input	Reliability	For any line typed at any prompt, including empty lines, lines of 10,000 characters and the end of input, CampusPlan shall never crash or print a stack trace. It shall either accept the input, print an "Error:" line, or (at the end of input) print "Goodbye." and exit with status 0.	All use cases	An automated test sends 1,000 random input lines to every prompt and checks the output and exit status

ID	Label	Quality	Requirement	Applies to	How we verify it
NFR-07	Responsive with a full term's data	Performance	On a typical student laptop (4 cores, 8 GB of RAM) with 200 students, 2,000 availability intervals, 100 events and the 7-room catalogue, at least 95% of menu actions shall finish within 1 second, and startup shall reach the main menu in under 3 seconds.	All use cases	A timed test runs 100 scripted actions against a generated dataset of that size
NFR-08	Runs offline on team machines	Portability	CampusPlan shall build and run with Java 21 on Windows, macOS and Linux with no network connection after the first build.	All use cases	./mvnw verify and a scripted demo run with networking turned off on at least one Windows and one non-Windows machine
NFR-09	Saved data stays readable	Compatibility	Saved data shall be UTF-8 JSON with a format version number, and dates and times shall be stored as entered, with no time zones. A file saved by an earlier iteration shall load in a later one with 100% of its records and IDs intact. A file with a newer version shall be refused without being modified.	UC-01, UC-07	P1 sample files are loaded by the P2 and P3 test suites, plus AC-01.17, AC-07.3 and a file copied between two operating systems
NFR-10	Same input, same output	Reliability	Given the same saved data and the same input, CampusPlan shall produce identical output in 10 out of 10 repeated runs. Rooms are listed by ID, students in the order they were added, and events by first date and then ID.	UC-02, UC-03, UC-06	An automated test runs the same scripted session 10 times and compares the output

ID	Label	Quality	Requirement	Applies to	How we verify it
NFR-11	Layered, documented code	Maintainability	The code shall be split into the ui, application, domain and persistence packages. Classes in domain shall not import from the other three packages or do any console or file input/output. Storage shall only be reached through repository interfaces. Every public class and method shall have Javadoc.	All use cases	A code review checklist on every pull request, and the Javadoc build reporting zero missing-comment warnings
NFR-12	Fully tested before merge	Testability	Every acceptance criterion in section 2 shall have at least one JUnit 5 test that names its ID. Before any merge to main, ./mvnw verify shall pass with 0 failures, run in under 2 minutes, and reach at least 80% line coverage on the domain and application packages.	All use cases	The test and coverage reports from ./mvnw verify, attached to each pull request
NFR-13	One-command build	Maintainability	From a fresh clone on a machine with only JDK 21 installed, ./mvnw verify (or mvnw.cmd verify on Windows) shall build the project and run every test using only the versions pinned in pom.xml.	All use cases	Two team members do a fresh clone and build at the end of each iteration
NFR-14	No real data or network use	Security	CampusPlan shall make no network connections while it runs, and every sample dataset shall use only fictional names and example email domains such as campus.example.	All use cases	A code search finds no java.net usage, and the sample data is reviewed